

Maps

A Map is a built-in data type that associates a unique Key with a specific Value. It provides lightning-fast lookups, updates, and deletions.

1. Declaring and Creating Maps

To define a map, you must specify the data type for both the **Key** and the **Value** inside square brackets:

There are two main ways to actually create the map in memory:

- **Using make():** This creates an empty map ready to accept data.

```
var idMap map[string]int
idMap = make(map[string]int)
```

- **Using a Map Literal:** If you already know some of the data, you can create and populate the map in a single step using curly brackets and colons.

```
idMap := map[string]int{"Joe": 123, "Jane": 456}
```

2. Reading, Writing, and Deleting

Interacting with a map looks very similar to an array, but instead of using a numerical index (like [0]), you use the exact Key.

- **Reading a Value:**

```
fmt.Println(idMap["Joe"]) // Prints 123
```

- **Adding or Updating a Value:**

```
idMap["Jane"] = 789
```

- If the key "Jane" doesn't exist yet, Go creates it.
- If the key "Jane" already exists, Go overwrites her old value with **789**.

- **Deleting a Value:** Use the built-in delete function.

```
delete(idMap, "Joe")
```

3. The "Existence Check" (Two-Value Assignment)

Because Go map keys default to their "**Zero Value**" if they don't exist (e.g., returning a 0 if you search for a missing integer), you sometimes need to know: Did this return 0 because the user's score is actually 0, or because the user doesn't exist in the map at all?

Go solves this by letting you return **two** values from a map lookup: the value itself, and a boolean confirming if the key exists.

```
id, isPresent := idMap["John"]
```

- If Joe is in the map, **isPresent** will be **true**. If not, it will be **false**.

4. Sizing and Iterating Over Maps

- **Length:** Just like arrays, you can use `len(idMap)` to instantly find out how many total key-value pairs are currently stored in the map.
- **Iteration:** You use the exact same `for...range` loop we used for slices, but instead of returning the Index and Value, `range` returns the **Key** and the **Value**.

```
for key, value := range idMap {  
    fmt.Println(key, value)  
}
```

Expert Takeaway:

While building a web application, Maps are your primary tool for handling **JSON** data. If a frontend sends a complex **JSON** payload of user preferences, Go's standard library will automatically **unmarshal** that payload directly into a **`map[string]interface{}`**, allowing you to quickly query specific configuration keys in **constant O(1) time** before saving the data to your database.

Remember: When you use a **`for...range`** loop on a Go **map**, the order it prints out is intentionally randomized by the compiler! You can never rely on a map keeping its items in alphabetical order.